

Second Largest Element

- Find the second largest element in a List<Integer>

```
package Collections_test;
import java.util.*;
public class Sec_largest_array_list {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        List<Integer> numbers = Arrays.asList(10, 45, 2, 33, 45, 90, 8);
        Collections.sort(numbers);
        System.out.println("List: "+numbers);
        System.out.println("2nd Lasrgest number: "+numbers.get(numbers.size()-2));
    }
}
List: [2, 8, 10, 33, 45, 45, 90]
2nd Lasrgest number: 45
```

Common Elements Between Two Sets

- Find intersection of two HashSet<Integer>.

```
package Collections_test;
import java.util.*;
public class Intersection_hash_set {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Set<Integer> num1=new HashSet<Integer>(Arrays.asList(1, 2, 3, 4));
        Set<Integer> num2=new HashSet<Integer>(Arrays.asList(4, 5, 6, 3));

        num1.retainAll(num2);
        System.out.println("Common Elements: " + num1);
    }
}
Common Elements: [3, 4]
```

Symmetric Difference

- Find elements present in either of the sets but not both.

```
package Collections_test;
import java.util.*;
public class Symmetric_diff {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Set<Integer> set1 = new HashSet<>(Arrays.asList(1, 2, 3, 4));
        Set<Integer> set2 = new HashSet<>(Arrays.asList(3, 4, 5, 6));
        Set<Integer> all = new HashSet<>(set1);
        all.addAll(set2);
        System.out.println("all Elements: " + all);
    }
}
```

```
        Set<Integer> common = new HashSet<>(set1);
        common.retainAll(set2);
        System.out.println("Common Elements: " + common);

        all.removeAll(common);
        System.out.println("Symmetric Elements: " + all);
    }
}
```

all Elements: [1, 2, 3, 4, 5, 6]
Common Elements: [3, 4]
Symmetric Elements: [1, 2, 5, 6]